Resolving Critical Memory and Concurrency Issues

Enhancing Codebase Stability and Performance

Part 1: Structural & Safety Fixes (Bug Resolution)

The original codebase exhibited several fatal memory and concurrency flaws when processing large datasets or utilizing modern CPU architectures. These issues were resolved through targeted bug resolutions.

Resolved Unconstrained Multithreading and Memory Corruption

The Issue

A critical observation of the original code was that it yielded varying, non-deterministic outputs, even when explicitly restricted to a single thread. This occurred because the OpenMP directive in the final postProcessIt phase (#pragma omp parallel for) lacked a thread-limit constraint. Consequently, the program silently spawned threads across all available CPU cores during post-processing, leading to severe memory collision and heap corruption as threads simultaneously attempted to write to a shared std::vector using push_back(). Further compounding the issue, all threads were mutating a shared Minimum Spanning Tree (mstCopy) simultaneously in the main loop.

The Fix

Variables such as the MST, visited lists, and single routes were scoped to be thread-local private copies. A #pragma omp critical lock was introduced when updating the global minCost score. Finally, push_back collisions were eliminated by pre-allocating the vector size and indexing directly, ensuring true thread safety.

Prevented Massive Stack Overflows

The Issue

The ShortCircutTour function utilized a recursive Depth-First Search (DFS). On large instances (e.g., a 6001-node graph), the function recursively called itself over 6,000 times, breaching the operating system's default stack limit and triggering a Segmentation Fault.

The Fix

The DFS was rewritten into an iterative format utilizing a std::stack. This safely uses heap memory to traverse the graph, allowing it to process massive files without crashing, while traversing nodes in the exact same reverse-iterator order as the original recursive function.

Plugged Memory Leaks

The Issue

The verify_sol function utilized the C-style malloc() function to allocate raw memory on the heap but lacked the corresponding free() command. This caused the program to progressively leak RAM during the verification phase.

The Fix

The raw malloc() allocation was replaced with a modern, auto-cleaning std::vector, which automatically handles its own memory deallocation upon exiting the function's scope.

Code Consolidation (DRY Principle)

The Issue

The logic for calculating the distance of a complete route was manually duplicated in five different locations throughout the file.

The Fix

A single calRouteValue utility function was created, standardizing the distance mathematics across the entire program and improving maintainability.

Part 2: Performance Enhancements

To accelerate the program without deviating from the original mathematical logic, hardware-friendly optimizations were applied.

Eliminated the rand() Parallel Bottleneck

The Optimization

The original code utilized the standard C rand() function inside the parallel OpenMP loop. In many standard library implementations, rand() relies on a hidden global mutex lock to protect its internal state. Consequently, when multiple threads called rand() simultaneously, they were forced to serialize (wait in line), effectively destroying the parallel speedup. This bottleneck was eliminated by assigning a private, thread-local std::default_random_engine to each thread. This allows true concurrent execution without lock contention, while dynamically seeding via the system clock and thread ID to maintain intended heuristic variance safely.

Flattened the Distance Matrix (Cache Optimization)

The Optimization

The 2D distance calculations were flattened into a single, contiguous 1D array (distanceMatrix[i * size + j]). Modern CPUs load contiguous memory into their L1 cache highly efficiently, drastically reducing the latency required to fetch the distance between two nodes.

Part 3: The Optimization Dilemma (Speed vs. Parity)

During the refactoring process, a significant dilemma emerged regarding speed versus parity. Several advanced algorithmic techniques could have reduced execution time exponentially. However, these techniques were intentionally excluded to preserve strict mathematical parity with the original codebase.

The 2-Opt Evaluation (O(1) Delta vs. O(N) Recalculation)

The Dilemma

When the 2-Opt algorithm swaps two roads, it only alters 4 edges. The optimal approach is a "Delta Evaluation"—simply evaluating if the 2 new edges are shorter than the 2 old edges. However, floating-point arithmetic is not perfectly associative. Subtracting 4 edges yields a microscopically different decimal than recalculating the sum of all 195 edges. This precision variance causes the Delta method to occasionally accept a route swap that the full-recalculation method rejects. To guarantee identical route selections, the slower, full-route recalculation was maintained.

Prim's Algorithm (O(E log V) Priority Queue vs. O(V^2) Array Search)

The Dilemma

Constructing the initial Minimum Spanning Tree via a Priority Queue drops the time complexity to O(E log V). However, if two distinct edges possess the exact same length, a linear array search breaks the tie based on sequential node ID, whereas a Priority Queue breaks ties based on binary heap structure. This results in a slightly different base tree, cascading into different heuristic shuffles. The $O(V^2)$ array search was retained for exact tie-breaking parity.

Mersenne Twister Randomization (std::mt19937)

The Dilemma

Upgrading to the modern std::mt19937 random engine provides a statistically superior distribution compared to the default engine. However, doing so completely alters the sequence of the 100,000 tree shuffles, exploring completely different paths than the original script. To match the expected output range, the default random engine was preserved.